

FORK TASK RUNNER C

Executor multiprocessos em C com fork, fila, lista, pilha, testes e CI

Brasil
2026

RESUMO

Este trabalho apresenta o Fork Task Runner C, um executor multiprocessos em C desenvolvido para demonstrar o uso de `fork()` em um fluxo real. O projeto integra fila FIFO, lista encadeada, pilha de eventos, criação de processos, coleta com `waitpid()`, `timeout`, testes automatizados e pipelines de qualidade. A proposta é aproximar conceitos de sistemas operacionais e estruturas de dados por meio de um repositório open source reproduzível, documentado e auditável.

Palavras-chave: C; fork; POSIX; escalonamento; estruturas de dados.

SUMÁRIO

- 1 Introdução
- 2 Fundamentação Técnica
- 3 Arquitetura Do Projeto
- 4 Algoritmo E Estruturas De Dados
- 5 Implementação E Uso
- 6 Testes, Qualidade E Segurança
- 7 Limitações
- 8 Conclusão
- Referências

1 INTRODUÇÃO

Exemplos introdutórios de `fork()` normalmente limitam-se a imprimir o identificador do processo pai e do processo filho. Embora esse tipo de demonstração seja útil para primeiro contato, ele não explica como processos são organizados em um fluxo de execução real. O Fork Task Runner C foi criado para preencher essa lacuna com um executor de tarefas simples, funcional e verificável.

O objetivo do projeto é demonstrar como estruturas de dados clássicas podem coordenar processos reais em C. A fila organiza tarefas pendentes, a lista encadeada acompanha processos ativos e a pilha registra eventos recentes do escalonador. O resultado é um projeto pequeno o suficiente para estudo e forte o suficiente para ser auditado por testes, documentação e CI.

2 FUNDAMENTAÇÃO TÉCNICA

A chamada `fork()` cria um novo processo a partir do processo chamador. O processo filho nasce com uma visão inicial semelhante à do pai, mas executa de forma independente. Em sistemas Unix e Linux modernos, a cópia de memória é otimizada por copy-on-write, reduzindo custo inicial de criação.

O padrão `fork()` seguido de `exec` é comum em shells e executores de comandos. O filho é criado com `fork()` e depois substitui sua imagem de processo pelo comando real. O pai permanece responsável por monitorar o ciclo de vida do filho e coletar seu status com `waitpid()`.

3 ARQUITETURA DO PROJETO

A arquitetura separa domínio, estruturas, aplicação, infraestrutura e apresentação. O processo pai é a única fonte de verdade do estado interno. Os filhos executam comandos isolados e não alteram diretamente a fila, a lista ou a pilha.

Camada	Responsabilidade
Domínio	Define tarefa, status e evento.
Estruturas	Implementa fila, lista encadeada e pilha.
Aplicação	Executa regras do scheduler FCFS com concorrência limitada.
Infraestrutura	Lê arquivos e executa chamadas POSIX de processo.
Apresentação	Renderiza painel terminal e relatório final.

4 ALGORITMO E ESTRUTURAS DE DADOS

O algoritmo escolhido é FCFS com concorrência limitada. A primeira tarefa que entra na fila é a primeira candidata à execução, desde que exista um slot livre dentro do limite `max_parallel`.

- O arquivo de tarefas é carregado para uma fila FIFO.
- O scheduler remove a próxima tarefa quando há capacidade.
- O pai cria um filho com `fork()`.
- O filho executa o comando via shell.
- O pai registra o PID na lista de processos ativos.
- O pai usa `waitpid()` para coletar conclusão.
- Eventos de início, sucesso, falha e timeout entram na pilha.

Estrutura	Uso	Complexidade Principal
Fila	Tarefas pendentes	Inserção e remoção $O(1)$
Lista encadeada	Processos ativos	Inserção $O(1)$, remoção por PID $O(n)$
Pilha	Eventos recentes	Inserção e remoção $O(1)$

5 IMPLEMENTAÇÃO E USO

O projeto pode ser compilado com CMake em Linux, macOS ou WSL 2. O Windows nativo não é suportado porque o projeto depende de chamadas POSIX.

```
git clone https://github.com/Victorzinn704/fork-task-runner-c.git
cd fork-task-runner-c
cmake -S . -B build -G Ninja -DFTR_ENABLE_STRICT=ON -DCMAKE_BUILD_TYPE=Debug
cmake --build build
ctest --test-dir build --output-on-failure
./build/fork-task-runner examples/tasks.sample.txt 2 --no-dashboard
```

O arquivo de tarefas usa o formato `id|nome|timeout_segundos|comando`. O comando fica no último campo para permitir pipelines POSIX com o caractere `|`.

6 TESTES, QUALIDADE E SEGURANÇA

O repositório possui testes unitários para as estruturas de dados, testes de integração para scheduler, falha e timeout, além de testes de CLI. A esteira de GitHub Actions executa CI com GCC e Clang, qualidade com `clang-format`, `clang-tidy`, `cppcheck`, `sanitizers`, `CodeQL`, `Valgrind` e `lint` de documentação.

O projeto executa comandos com `/bin/sh -c` e, por isso, não deve receber arquivos de tarefas não confiáveis. Essa limitação está documentada no arquivo `SECURITY.md`.

7 LIMITAÇÕES

- Não há suporte para Windows nativo.
- Não há sandbox de comandos.
- Não há prioridade, preempção ou memória compartilhada.
- O projeto não substitui ferramentas reais de CI ou orquestração.

8 CONCLUSÃO

O Fork Task Runner C cumpre seu objetivo acadêmico e técnico: demonstrar processos reais com `fork()` dentro de um fluxo organizado por estruturas de dados clássicas. A solução preserva baixa complexidade, possui documentação reproduzível e apresenta critérios de qualidade compatíveis com um repositório open source inicial.

REFERÊNCIAS

KERRISK, Michael. **fork(2) — Linux manual page**. Disponível em: <https://man7.org/linux/man-pages/man2/fork.2.html>. Acesso em: 29 jun. 2026.

KERRISK, Michael. **waitpid(2) — Linux manual page**. Disponível em: <https://man7.org/linux/man-pages/man2/waitpid.2.html>. Acesso em: 29 jun. 2026.

KERRISK, Michael. **execve(2) — Linux manual page**. Disponível em: <https://man7.org/linux/man-pages/man2/execve.2.html>. Acesso em: 29 jun. 2026.

THE OPEN GROUP. **fork — The Open Group Base Specifications**. Disponível em: <https://pubs.opengroup.org/onlinepubs/9799919799/functions/fork.html>. Acesso em: 29 jun. 2026.

REDIS. **Redis persistence**. Disponível em: https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/. Acesso em: 29 jun. 2026.

CMAKE. **CTest manual**. Disponível em: <https://cmake.org/cmake/help/latest/manual/ctest.1.html>. Acesso em: 29 jun. 2026.

GITHUB. **About code scanning with CodeQL**. Disponível em: <https://docs.github.com/en/code-security/code-scanning/introduction-to-code-scanning/about-code-scanning-with-codeql>. Acesso em: 29 jun. 2026.